

Bits, Bytes and Integers – Part 1

15-213/18-213/15-513: Introduction to Computer Systems
2nd Lecture, Aug. 31, 2017

Today's Instructor:

Randy Bryant

数据的表示和运算(chapter2, 2-4)

- 分三个部分介绍
 - 非数值数据的表示、数据的存储
 - 数据宽度单位
 - 硬件特征：大端/小端、对齐存放
 - 数值数据的表示
 - 定点数的编码表示
 - 整数的表示
 - 无符号整数、带符号整数
 - 浮点数的表示
 - C语言程序的整数类型和浮点数类型

数据的表示和运算

- 分三个部分介绍（续）

- 数据的运算

- 按位运算和逻辑运算
 - 移位运算
 - 位扩展和位截断运算
 - 无符号和带符号整数的加减运算
 - 无符号和带符号整数的乘除运算
 - 变量与常数之间的乘除运算

围绕C语言中的运算，解释其在底层机器级的实现方法

从高级语言程序中的表达式出发，用机器数在具体电路中的执行过程，来解释表达式的执行结果

Today: Bits, Bytes, and Integers

- **Representing information as bits**
- **Bit-level manipulations**
- **Integers**
 - Representation: unsigned and signed
 - Conversion, casting
 - Expanding, truncating
 - Addition, negation, multiplication, shifting
 - Summary
- **Representations in memory, pointers, strings**

Encoding Byte Values

■ Byte = 8 bits

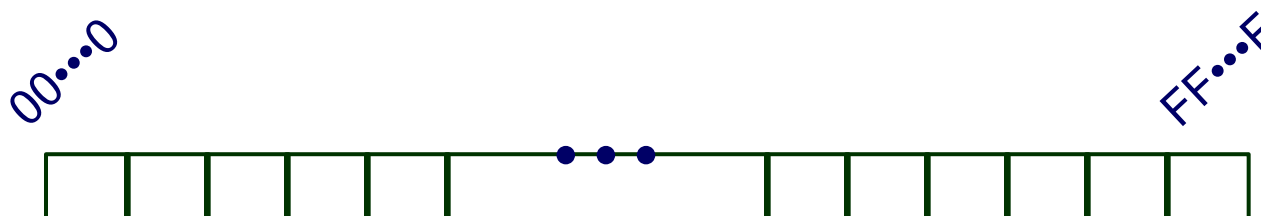
- Binary 00000000_2 to 11111111_2
- Decimal: 0_{10} to 255_{10}
- Hexadecimal 00_{16} to FF_{16}
 - Base 16 number representation
 - Use characters '0' to '9' and 'A' to 'F'
 - Write $FA1D37B_{16}$ in C as
 - `0xFA1D37B`
 - `0xfa1d37b`

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

15213: 0011 1011 0110 1101

3
B
6
D

Byte-Oriented Memory Organization



■ Programs refer to data by address

- Conceptually, envision it as a very large array of bytes
 - In reality, it's not, but can think of it that way
- An address is like an index into that array
 - and, a pointer variable stores an address

■ Note: system provides private address spaces to each “process”

- Think of a process as a program being executed
- So, a program can clobber its own data, but not that of others

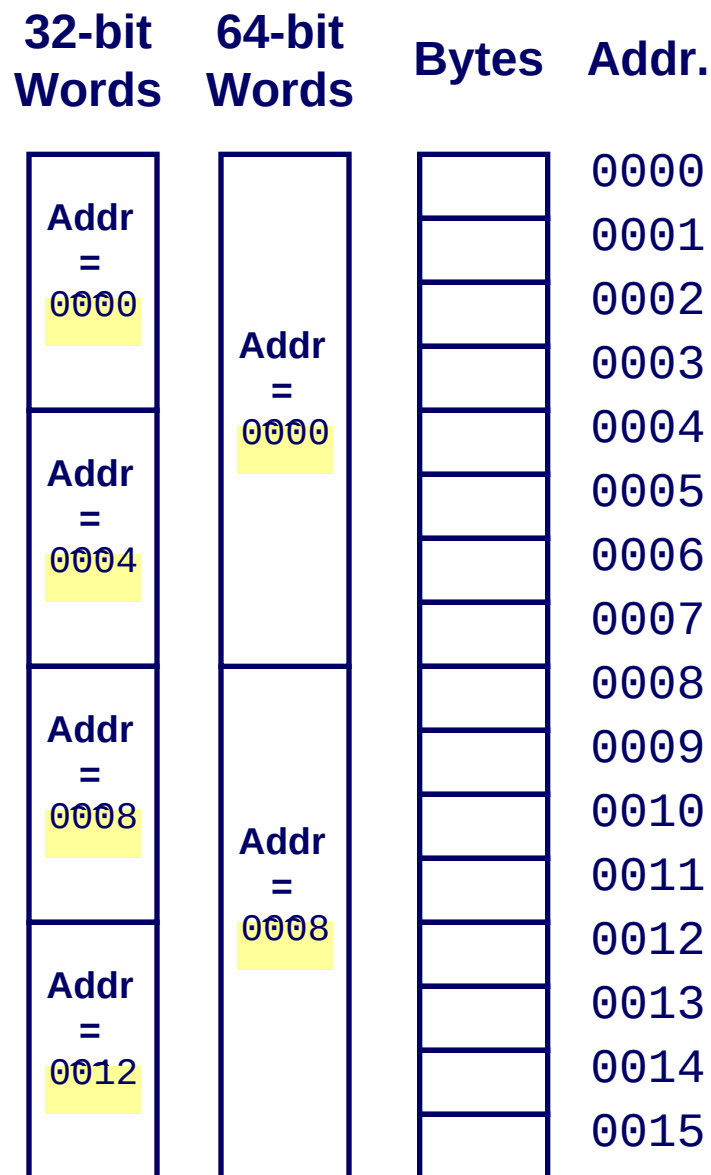
Machine Words

- **Any given computer has a “Word Size”**
 - Nominal size of integer-valued data
 - and of addresses
 - Until recently, most machines used 32 bits (4 bytes) as word size
 - Limits addresses to 4GB (2_{32} bytes)
 - Increasingly, machines have 64-bit word size
 - Potentially, could have 18 EB (exabytes) of addressable memory
 - That's $18.4 \times 10_{18}$
 - Machines still support multiple data formats
 - Fractions or multiples of word size
 - Always integral number of bytes

Word-Oriented Memory Organization

■ Addresses Specify Byte Locations

- Address of first byte in word
- Addresses of successive words differ by 4 (32-bit) or 8 (64-bit)



Example Data Representations

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
pointer	4	8	8

- So, how are the bytes within a multi-byte word ordered in memory?

数据的存储和排列顺序

- 80年代开始，几乎所有机器都用**字节编址**
- ISA设计时要考虑的两个问题：
 - 如何根据一个字节地址取到一个32位的字？ - **字的存放问题**
 - 一个字能否存放在任何字节边界？ - **字的边界对齐问题**

例如，若 $\text{int } i =$

-65535，存放在内存100号单元（即占100# ~ 103#），则用“取数”指令访问100号单元取出 i 时，必须清楚 i 的4个字节是如何存放的。

$$65535 = 2^{16} - 1$$

$$[-65535]_{\text{补}} = \text{FFFF0001H}$$

Word:

	FF 103	FF 102	00 101	01 100
msb				lsb
100	101	102	103	

little endian word 100#

big endian word 100#

大端方式 (Big Endian) : MSB所在的地址是数的地址

e.g. IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA

小端方式 (Little Endian) : LSB所在的地址是数的地址

e.g. Intel 80x86, DEC VAX

Representing Integers

Decimal: 15213

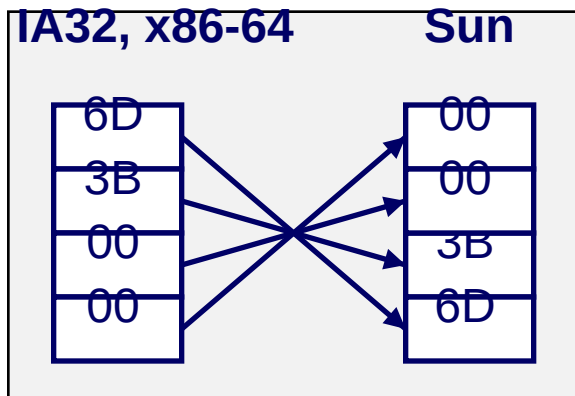
Binary: 0011 1011 0110 1101

Hex: 3 B 6 D

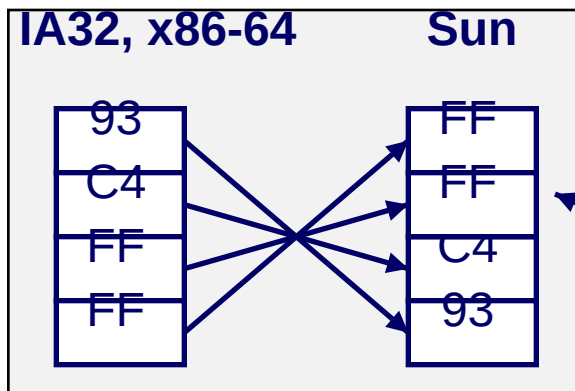
int A = 15213;

long int C = 15213;

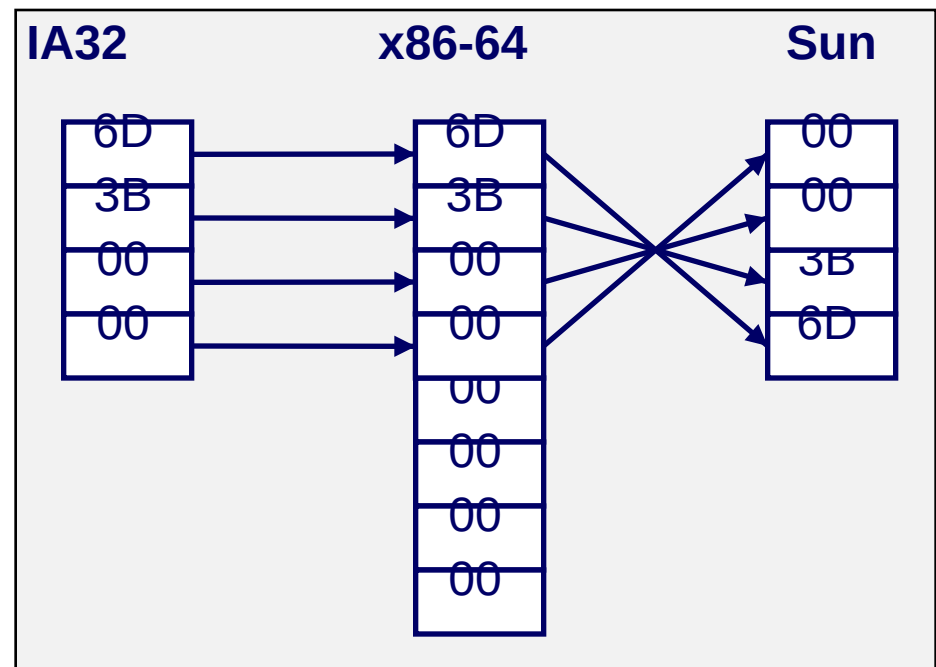
Increasing addresses
↓



int B = -15213;



Two's complement representation



BIG Endian versus Little Endian

Ex3: Memory layout of a instruction located in 1000

假定小端机器中指令: `mov AX, 0x12345(BX)`

其中操作码mov为40H, 寄存器AX和BX的编号分别为0001B和0010B, 立即数占32位, 则存放顺序为:



若在大端机器上, 则存放顺序如何?

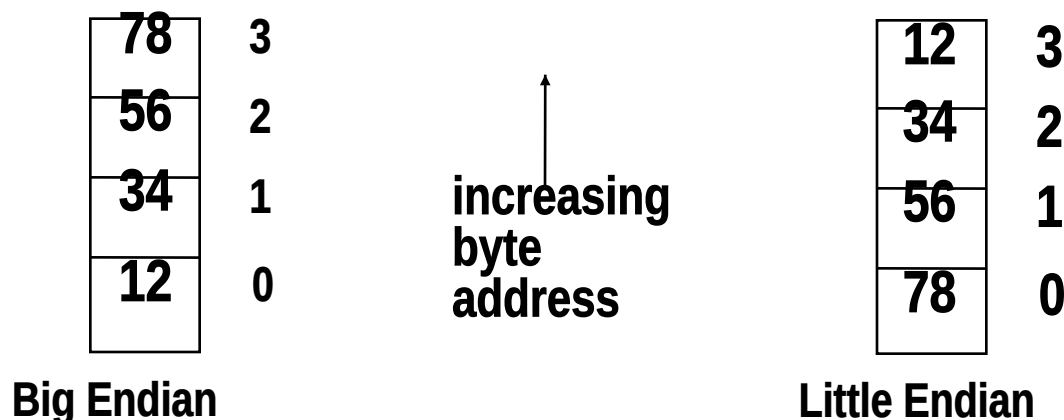


00	1005	45
01	1004	23
23	1003	01
45	1002	00
12	1001	12
40	1000	40

地址

只需要考虑指令中立即数的顺序!

Byte Swap Problem (字节交换问题)



上述存放在0号单元的数据（字）是什么？ **12345678H**？ **78563412H**？

存放方式不同的机器间程序移植或数据通信时，会发生什么问题？

- ◆ 每个系统内部是一致的，但在系统间通信时可能会发生问题！
- ◆ 因为顺序不同，需要进行顺序转换

音、视频和图像等文件格式或处理程序都涉及到字节顺序问题

ex. Little endian: GIF, PC Paintbrush, Microsoft RTF, etc

Big endian: Adobe Photoshop, JPEG, MacPaint, etc

Alignment(对齐)

Alignment: 要求数据的地址是相应的边界地址

- 目前机器字长一般为32位或64位，而存储器地址按字节编址
- 指令系统支持对字节、半字、字及双字的运算，也有位处理指令
- 各种不同长度的数据存放时，有两种处理方式：

- 按边界对齐 （假定存储字的宽度为32位，按字节编址）

- 字地址：4的倍数(低两位为0)

- 半字地址：2的倍数(低位为0)

- 字节地址：任意

每4个字节可同时读写



- 不按边界对齐

坏处：可能会增加访存次数！

Alignment(对齐)

如: int i, short k, double x, char c, short j,.....

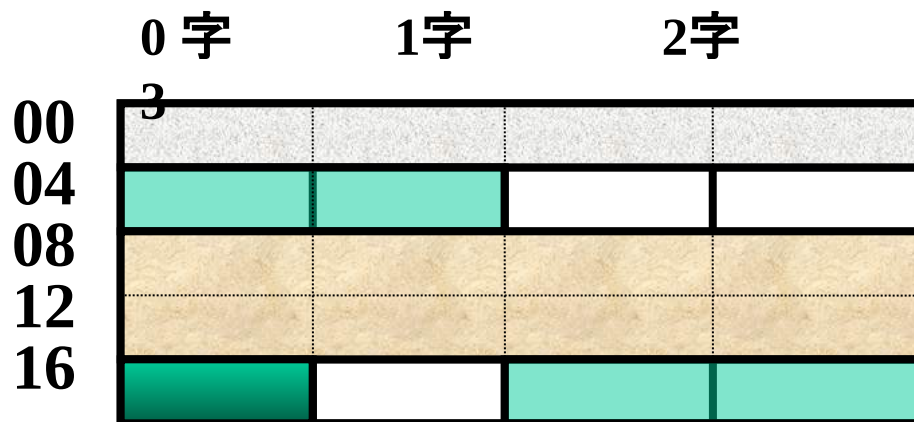
存储器按字节
编址

每次只能读写
某个字地址开
始的4个单元中
连续的1个、2个
个、3个或4个
字节

按边界对齐

x: 2个周期

j: 1个周期



&i=0; &k=4; &x=8; &c=16; &j=18;.....

字节0 字节1 字节2 字节3

虽节省了空间,
但增加了访存次
数!

边界不对齐

x: 3个周期

j: 2个周期



: &i=0; &k=4; &x=6; &c=14; &j=15;.....

需要权衡, 目前来
来看, 浪费一点存
存储空间没有关系!

Alignment(对齐) 举例

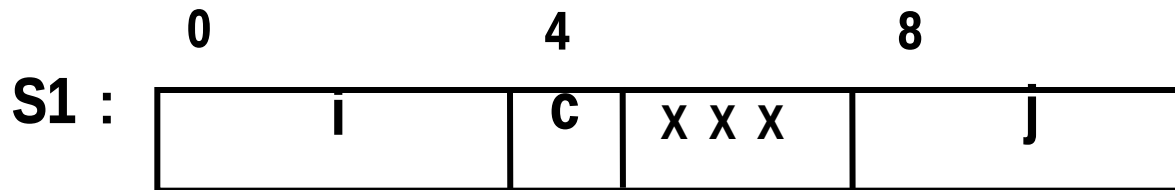
例如，考虑下列两个结构声明：

```
struct S1 {
    int    i;
    char   c;
    int    j;
};
```

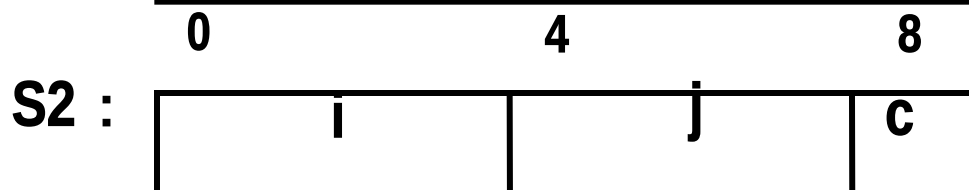
```
struct S2 {
    int    i;
    int    j;
    char   c;
};
```

在要求对齐的情况下，哪种结构声明更好？

S2比S1好



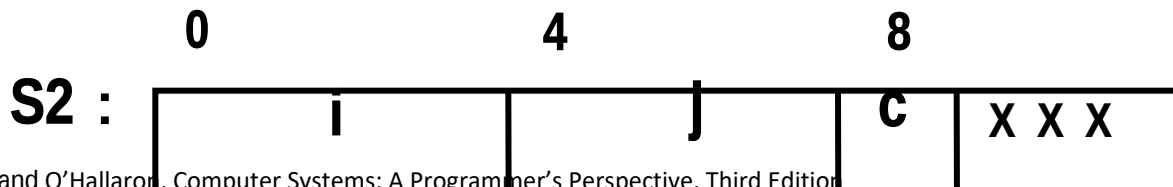
需要12个字节



只需要9个字节

对于“struct S2 d[4]”，只分配9个字节能否满足对齐要求？

不能！



也需要12个字节

Today: Bits, Bytes, and Integers

- Representing information as bits
- **Bit-level manipulations**
- **Integers**
 - Representation: unsigned and signed
 - Conversion, casting
 - Expanding, truncating
 - Addition, negation, multiplication, shifting
 - Summary
- Representations in memory, pointers, strings

逻辑数据的编码表示

- 表示

- 用一位表示 。例如，真：1 / 假：0
- N位二进制数可表示N个逻辑数据，或一个位串

- 运算

- 按位进行
- 如:按位与 / 按位或 / 逻辑左移 / 逻辑右移 等

- 识别

- 逻辑数据和数值数据在形式上并无差别，也是一串0/1序列，机器靠指令来识别。

- 位串

- 用来表示若干个状态位或控制位（OS中使用较多）

例如，x86

				OF	DF	IF	TF	SF	ZF		AF		PF		CF
--	--	--	--	----	----	----	----	----	----	--	----	--	----	--	----

Boolean Algebra

■ Developed by George Boole in 19th Century

- Algebraic representation of logic
 - Encode “True” as 1 and “False” as 0

And

- $A \& B = 1$ when both $A=1$ and $B=1$

$\&$	0	1
0	0	0
1	0	1

Not

- $\sim A = 1$ when $A=0$

$\sim$	
0	1
1	0

Or

- $A | B = 1$ when either $A=1$ or $B=1$

$ $	0	1
0	0	1
1	1	1

Exclusive-Or (Xor)

- $A \wedge B = 1$ when either $A=1$ or $B=1$, but not both

$\wedge$	0	1
0	0	1
1	1	0

Bit-Level Operations in C

■ Operations `&`, `|`, `~`, `^` Available in C

- Apply to any “integral” data type
 - long, int, short, char, unsigned
- View arguments as bit vectors
- Arguments applied bit-wise

■ Examples (Char data type)

- `~0x41` →
- `~0x00` →
- `0x69 & 0x55` →
- `0x69 | 0x55` →

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Bit-Level Operations in C

■ Operations $\&$, $|$, $\sim$, $\wedge$ Available in C

- Apply to any “integral” data type
 - long, int, short, char, unsigned
- View arguments as bit vectors
- Arguments applied bit-wise

■ Examples (Char data type)

- $\sim 0x41 \rightarrow 0xBE$
 - $\sim 0100\ 0001_2 \rightarrow 1011\ 1110_2$
- $\sim 0x00 \rightarrow 0xFF$
 - $\sim 0000\ 0000_2 \rightarrow 1111\ 1111_2$
- $0x69 \& 0x55 \rightarrow 0x41$
 - $0110\ 1001_2 \& 0101\ 0101_2 \rightarrow 0100\ 0001_2$
- $0x69 | 0x55 \rightarrow 0x7D$
 - $0110\ 1001_2 | 0101\ 0101_2 \rightarrow 0111\ 1101_2$

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Contrast: Logic Operations in C

■ Contrast to Bit-Level Operators

■ Logic Operations: `&&`, `||`, `!`

- View 0 as “False”

- Anything nonzero

- Always

- Early

■ Example

- `!0x41` →

- `!0x00` →

- `!!0x41` → `0x01`

- `0x69 && 0x55` → `0x01`

- `0x69 || 0x55` → `0x01`

- `p && *p` (avoids null pointer access)

**Watch out for `&&` vs. `&` (and `||` vs. `|`)...
one of the more common oopsies in
C programming**

Shift Operations

■ Left Shift: $x \ll y$

- Shift bit-vector x left y positions
 - Throw away extra bits on left
 - Fill with 0's on right

■ Right Shift: $x \gg y$

- Shift bit-vector x right y positions
 - Throw away extra bits on right
- Logical shift**
 - Fill with 0's on left
- Arithmetic shift**
 - Replicate most significant bit on left

■ Undefined Behavior

- Shift amount < 0 or $\geq$ word size

Argument x	0 1100010
$\ll 3$	00010000
Log. $\gg 2$	00011000
Arith. $\gg 2$	00011000

Argument x	1 0100010
$\ll 3$	00010000
Log. $\gg 2$	00101000
Arith. $\gg 2$	11101000

西文字符的编码表示

• 特点

- 是一种拼音文字，用有限几个字母可拼写出所有单词
- 只对有限个字母和数学符号、标点符号等辅助字符编码
- 所有字符总数不超过256个，使用7或8个二进位可表示

• 表示（常用编码为7位ASCII码）

- 十进制数字：0/1/2.../9（30H）
 - 英文字母：A/B/.../Z/a/b/.../z（41H、61H）
 - 专用符号：+/-/%/*/&/.....
 - 控制字符（不可打印或显示）（回车：0DH；换行：0AH）
- } 必须熟悉对应的ASCII码！

• 操作

- 字符串操作，如：传送/比较 等

Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
 - **Representation: unsigned and signed**
 - Conversion, casting
 - Expanding, truncating
 - Addition, negation, multiplication, shifting
 - Summary
- Representations in memory, pointers, strings
- Summary

Encoding Integers

Unsigned

$$\text{BINARY } x = \sum_{i=0}^{w-1} x_i 2^i$$

Two's Complement

$$\text{BINARY } x = \sum_{i=0}^{w-1} x_i 2^i$$

Sign Bit

```
short int x = 15213;
short int y = -15213;
```

■ C short 2 bytes long

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
y	-15213	C4 93	11000100 10010011

■ Sign Bit

- For 2's complement, most significant bit indicates sign
 - 0 for nonnegative
 - 1 for negative

Two-complement: Simple Example

$$\begin{array}{rcccccl}
 & & & 8 & 4 & 2 & 1 \\
 & & - & & & & \\
 10 & = & 16 & & & & \\
 & & 0 & 1 & 0 & 1 & 0
 \end{array}$$

$$8 + 2 = 10$$

$$\begin{array}{rcccccl}
 & & & 8 & 4 & 2 & 1 \\
 & & - & & & & \\
 -10 & = & 16 & & & & \\
 & & 1 & 0 & 1 & 1 & 0
 \end{array}$$

$$-16 + 4 + 2 = -10$$

Two-complement Encoding Example (Cont.)

x = 15213: 00111011 01101101
y = -15213: 11000100 10010011

Weight	15213		-15213	
1	1	1	1	1
2	0	0	1	2
4	1	4	0	0
8	1	8	0	0
16	0	0	1	16
32	1	32	0	0
64	1	64	0	0
128	0	0	1	128
256	1	256	0	0
512	1	512	0	0
1024	0	0	1	1024
2048	1	2048	0	0
4096	1	4096	0	0
8192	1	8192	0	0
16384	0	0	1	16384
-32768	0	0	1	-32768
Sum	15213		-15213	

Numeric Ranges

■ Unsigned Values

- $UMin = 0$
000...0
- $UMax = 2_w - 1$
111...1

■ Two's Complement Values

- $TMin = -2_{w-1}$
100...0
- $TMax = 2_{w-1} - 1$
011...1
- Minus 1
111...1

Values for $W = 16$

	Decimal	Hex	Binary
UMax	65535	FF FF	11111111 11111111
TMax	32767	7F FF	01111111 11111111
TMin	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

Values for Different Word Sizes

	W			
	8	16	32	64
UMax	255	65,535	4,294,967,295	18,446,744,073,709,551,615
TMax	127	32,767	2,147,483,647	9,223,372,036,854,775,807
TMin	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808

■ Observations

- $|TMin| = TMax + 1$
 - Asymmetric range
- $UMax = 2 * TMax + 1$

■ C Programming

- `#include <limits.h>`
- Declares constants, e.g.,
 - `ULONG_MAX`
 - `LONG_MAX`
 - `LONG_MIN`
- Values platform specific

Unsigned & Signed Numeric Values

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

■ Equivalence

- Same encodings for nonnegative values

■ Uniqueness

- Every bit pattern represents unique integer value
- Each representable integer has unique bit encoding

■ ⇒ Can Invert Mappings

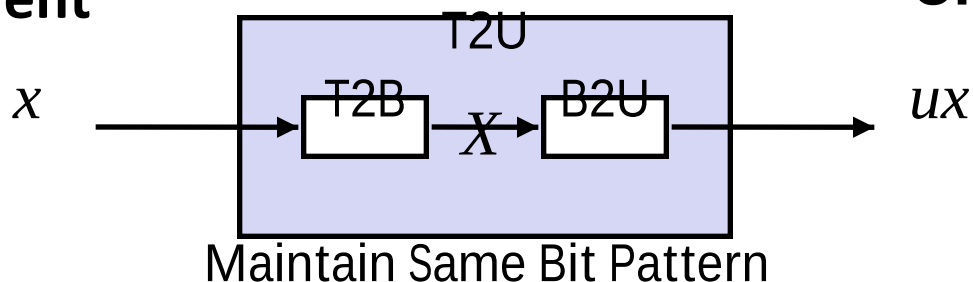
- $U2B(x) = B2U_{-1}(x)$
 - Bit pattern for unsigned integer
- $T2B(x) = B2T_{-1}(x)$
 - Bit pattern for two's comp integer

Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
 - Representation: unsigned and signed
 - **Conversion, casting**
 - Expanding, truncating
 - Addition, negation, multiplication, shifting
 - Summary
- Representations in memory, pointers, strings

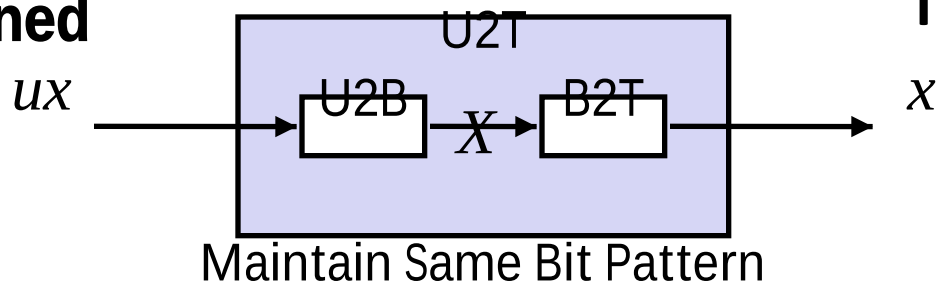
Mapping Between Signed & Unsigned

Two's Complement



Unsigned

Unsigned



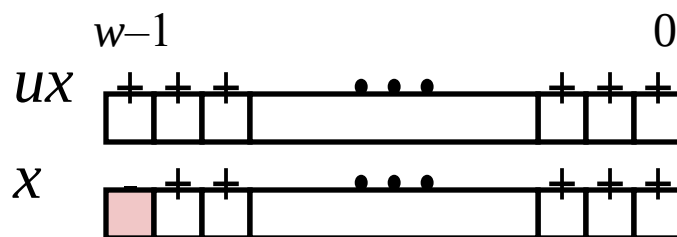
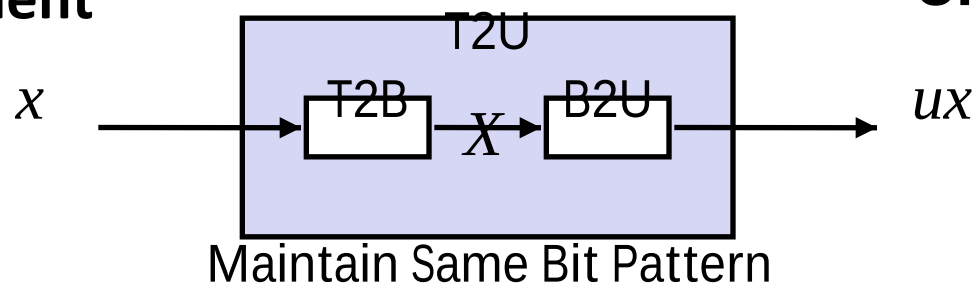
Two's Complement

- Mappings between unsigned and two's complement numbers:
Keep bit representations and reinterpret

Relation between Signed & Unsigned

Two's Complement

Unsigned



Large negative weight
becomes
Large positive weight

Signed vs. Unsigned in C

■ Constants

- By default are considered to be **signed integers**
- Unsigned if have “**U**” as suffix

0U, 4294967259U

■ Casting

- **Explicit casting** between signed & unsigned same as U2T and T2U

```
int tx, ty;  
unsigned ux, uy;  
tx = (int) ux;  
uy = (unsigned) ty;
```

- **Implicit casting** also occurs via assignments and procedure calls

```
tx = ux;                                int fun(unsigned u);  
uy = ty;                                uy = fun(tx);
```

Casting Surprises

■ Expression Evaluation

- If there is a mix of unsigned and signed in single expression, ***signed values implicitly cast to unsigned***
- Including comparison operations `<`, `>`, `==`, `<=`, `>=`
- Examples for $W = 32$: **`TMIN = -2,147,483,648`**, **`TMAX = 2,147,483,647`**

关系表达式	运算类型	结果	说明
<code>0 == 0U</code> <code>-1 < 0</code> <code>-1 < 0U</code> <code>2147483647 > -2147483647-1</code> <code>2147483647U > -2147483647-1</code> <code>2147483647 > (int) 2147483648U</code> <code>-1 > -2</code> <code>(unsigned) -1 > -2</code>			

C语言程序中的整数

关系 表达式	类型	结果	说明
<code>0 == 0U</code>	无	1	<code>00...0B = 00...0B</code>
<code>-1 < 0</code>	带	1	<code>11...1B (-1) < 00...0B (0)</code>
<code>-1 < 0U</code>	无	0*	<code>11...1B (2₃₂-1) > 00...0B(0)</code>
<code>2147483647 > -2147483647 - 1</code>	带	1	<code>011...1B (2₃₁-1) > 100...0B (-2₃₁)</code>
<code>2147483647U > -2147483647 - 1</code>	无	0*	<code>011...1B (2₃₁-1) < 100...0B(2₃₁)</code>
<code>2147483647 > (int) 2147483648U</code>	带	1*	<code>011...1B (2₃₁-1) > 100...0B (-2₃₁)</code>
<code>-1 > -2</code>	带	1	<code>11...1B (-1) > 11...10B (-2)</code>
<code>(unsigned) -1 > -2</code>	无	1	<code>11...1B (2₃₂-1) > 11...10B (2₃₂-2)</code>

带*的结果与常规预想的相反！

C语言程序中的整数

例如，考虑以下C代码：

```
1    int x = -1;
2    unsigned u = 2147483648;
3
4    printf ( "x = %u = %d\n" , x, x);
5    printf ( "u = %u = %d\n" , u, u);
```

在32位机器上运行上述代码时，它的输出结果是什么？为什么？

$x = 4294967295 = -1$

$u = 2147483648 = -2147483648$



因为-1的补码整数表示为“11...1”，作为32位无符号数解释时，其值为 $2_{32}-1 = 4\ 294\ 967\ 296-1 = 4\ 294\ 967\ 295$ 。



2_{31} 的无符号数表示为“100...0”，被解释为32位带符号整数时，其值为最小负数： $-2_{32-1} = -2_{31} = -2\ 147\ 483\ 648$ 。

C语言程序中的整数

- 1) 在有些32位系统上, C表达式 $-2147483648 < 2147483647$ 的执行结果为false。Why?
 - 2) 若定义变量 `int i=-2147483648;`, 则 `i < 2147483647` 的执行结果为true。Why?
 - 3) 如果将表达式写成 `-2147483647-1 < 2147483647`, 则结果会怎样呢? Why?
-
- 1) 在ISO C90 准下, 2147483648为unsigned类型, 因此 `-2147483648 < 2147483647` 按无符号数比较, 10.....0B比01.....1大, 结果为false。
在ISO C99 准下, 2147483648为int类型, 因此 `-2147483648 < 2147483647` 按带符号整数比较, 10.....0B比01.....1小, 结果为true。
 - 2) `i < 2147483647` 按int型数比较, 结果为true。
 - 3) `-2147483647-1 < 2147483647` 按int型比较, 结果为true。

Unsigned vs. Signed: Easy to Make Mistakes

```
unsigned i;  
for (i = cnt-2; i >= 0; i--)  
    a[i] += a[i+1];
```

- Can be very subtle

```
#define DELTA sizeof(int)  
int i;  
for (i = CNT; i-DELTA >= 0; i-= DELTA)  
    . . .
```


Summary

Casting Signed $\leftrightarrow$ Unsigned: Basic Rules

- Bit pattern is maintained
- But reinterpreted
- Can have unexpected effects: adding or subtracting 2_w
- Expression containing signed and unsigned int
 - `int` is cast to `unsigned`!!

Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
 - Representation: unsigned and signed
 - Conversion, casting
 - **Expanding, truncating**
 - Addition, negation, multiplication, shifting
 - Summary
- Representations in memory, pointers, strings

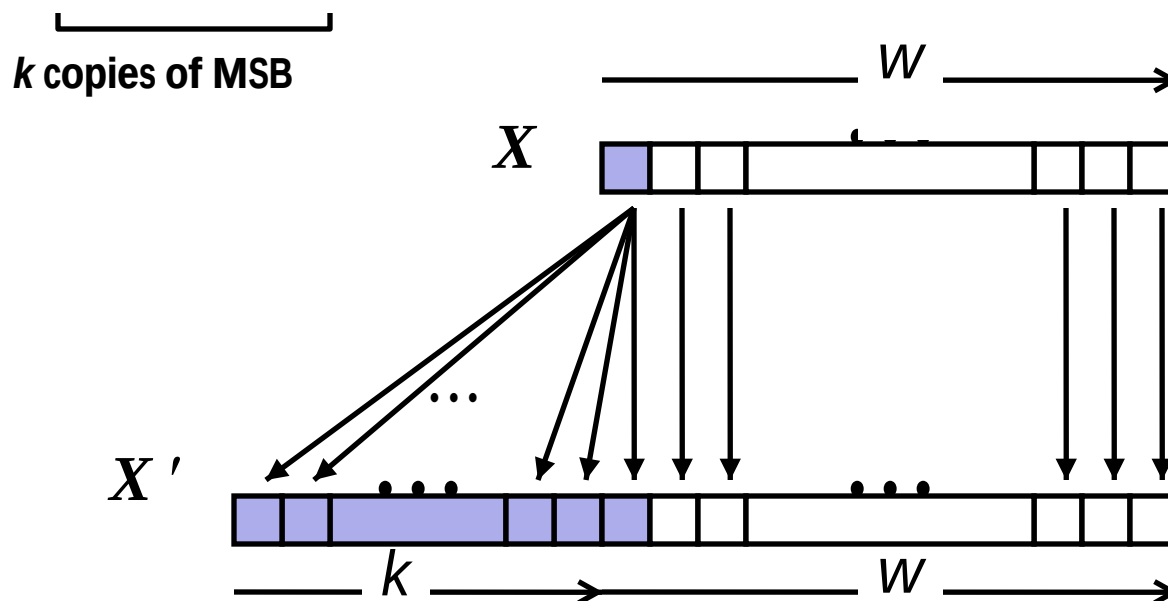
Sign Extension

■ Task:

- Given w -bit signed integer x
- Convert it to $w+k$ -bit integer with same value

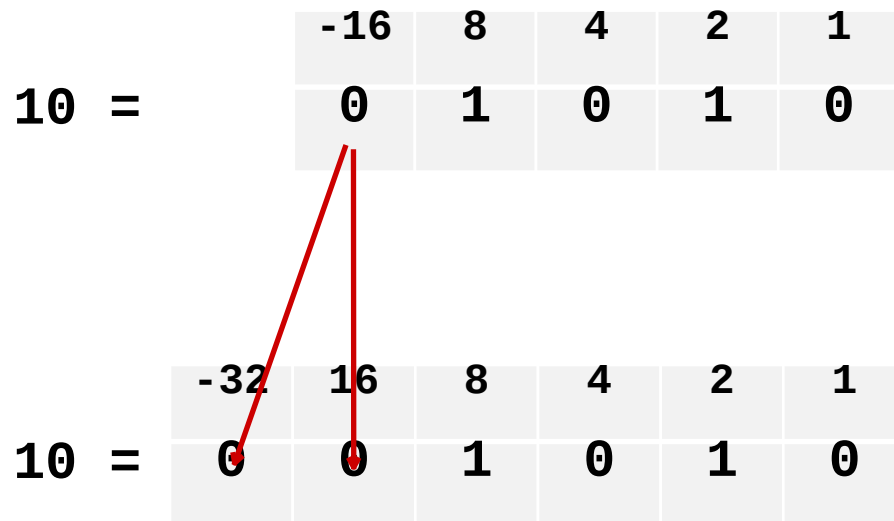
■ Rule:

- Make k copies of sign bit:
- $X' = X_{w-1}, \dots, X_{w-1}, X_{w-1}, X_{w-2}, \dots, X_0$

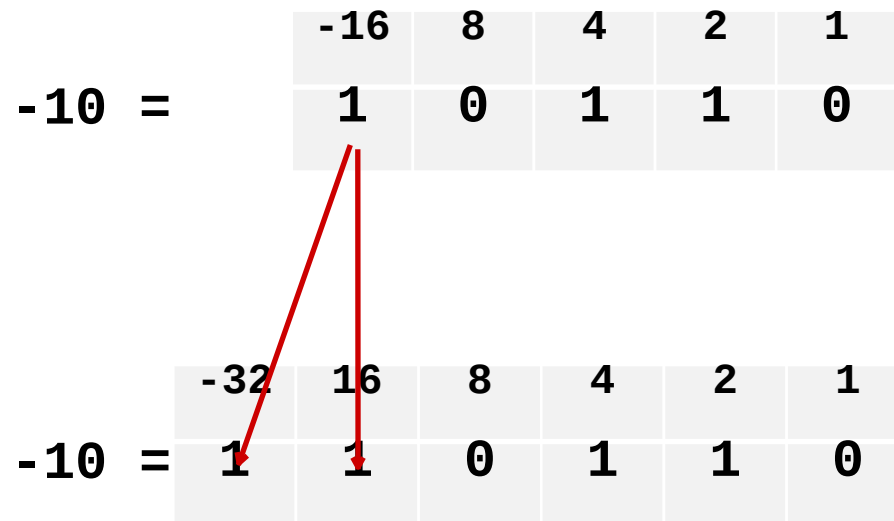


Sign Extension: Simple Example

Positive number



Negative number



Larger Sign Extension Example

```
short int x = 15213;
int      ix = (int) x;
short int y = -15213;
int      iy = (int) y;
```

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
ix	15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y	-15213	C4 93	11000100 10010011
iy	-15213	FF FF C4 93	11111111 11111111 11000100 10010011

- Converting from smaller to larger integer data type
- C automatically performs sign extension

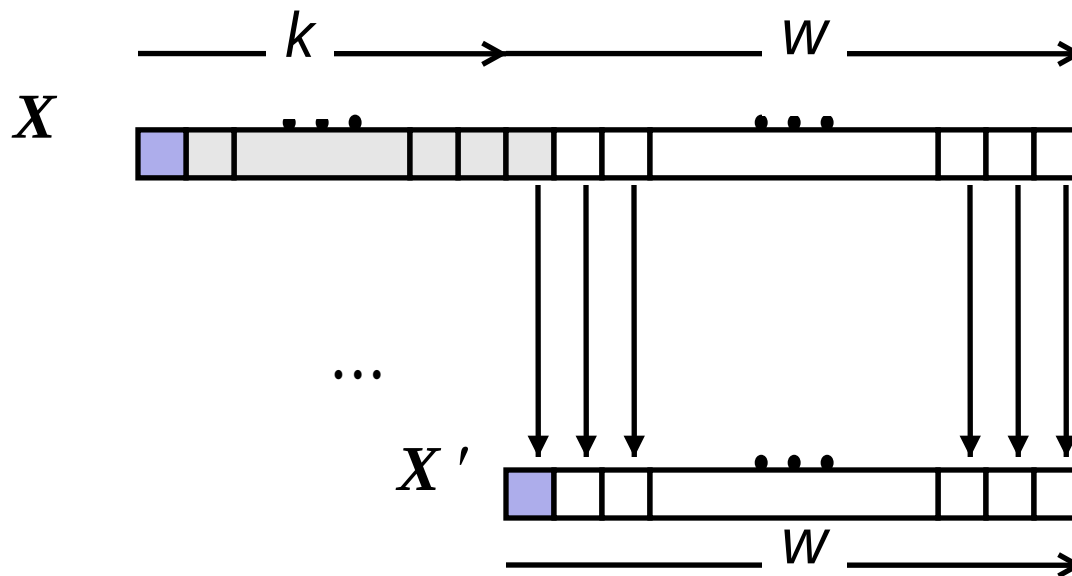
Truncation

■ Task:

- Given $k+w$ -bit signed or unsigned integer X
- Convert it to w -bit integer X' with same value for “small enough” X

■ Rule:

- Drop top k bits:
- $X' = X_{w-1}, X_{w-2}, \dots, X_0$



Truncation: Simple Example

No sign change

$$2 =$$

-16	8	4	2	1
0	0	0	1	0

$$2 =$$

-8	4	2	1
0	0	1	0

$2 \bmod 16 = 2$

$$-6 =$$

-16	8	4	2	1
1	1	0	1	0

$$-6 =$$

-8	4	2	1
1	0	1	0

$-6 \bmod 16 = 26 \bmod 16 = 10 \bmod 16 = -6$

Sign change

$$10 =$$

-16	8	4	2	1
0	1	0	1	0

$$-6 =$$

-8	4	2	1
1	0	1	0

$10 \bmod 16 = 10 \bmod 16 = 10 \bmod 16 = -6$

$$-10 =$$

-16	8	4	2	1
1	0	1	1	0

$$6 =$$

-8	4	2	1
0	1	1	0

$-10 \bmod 16 = 22 \bmod 16 = 6 \bmod 16 = 6$

例1（扩展操作）： 在大端机上输出s

i, usi, i,

ui的十进制和十六进制值是什么？

```
short si = -32768;
```

```
unsigned short usi = si;
```

```
int i = si;
```

```
unsigned ui = usi ;
```

```
si = -32768    80 00
```

```
usi = 32768    80 00
```

```
i = -32768    FF FF 80 00
```

```
ui = 32768    00 00 80 00
```

例2（截断操作）： i和j是否相等？

```
int i = 32768;
```

```
short si = (short) i;
```

```
int j = si;
```

不相等！

```
i = 32768    00 00 80 00
```

```
si = -32768    80 00
```

```
j = -32768    FF FF 80 00
```

原因： 对i截断时发生了“溢出”，即：32768截断为16位数时，因其超出16位能表示的最大值，故无法截断为正确的16位数！

Summary:

Expanding, Truncating: Basic Rules

■ Expanding (e.g., short int to int)

- Unsigned: zeros added
- Signed: sign extension
- Both yield expected result

■ Truncating (e.g., unsigned to unsigned short)

- Unsigned/signed: bits are truncated
- Result reinterpreted
- Unsigned: mod operation
- Signed: similar to mod
- For small (in magnitude) numbers yields expected behavior

小结

- 非数值数据的表示
 - 逻辑数据用来表示真/假或N位位串，按位运算
 - 西文字符：用ASCII码表示
 - 汉字编码
- 数据的宽度
 - 位、字节、字（不一定等于字长）
 - k / K / M / G / T / P / E / Z / Y 有不同的含义
- 数据的存储排列
 - 数据的地址：连续若干单元中最小的地址，即：从小地址开始存放数据
 - 问题：若一个short型数据si存放在单元0x08000100和0x08000101中，那么si的地址是什么？
 - 大端方式：用MSB存放的地址表示数据的地址
 - 小端方式：用LSB存放的地址表示数据的地址
 - 按边界对齐可减少访存次数

小结

- 在机器内部编码后的数称为机器数，其值称为真值
- 定义数值数据有三个要素：进制、定点/浮点、编码
- 整数的表示
 - 无符号数：正整数，用来表示地址等；带符号整数：用补码表示
- C语言中的整数
 - 无符号数：unsigned int (short / long)；带符号数： int (short / long)
- 十进制数的表示：用ASCII码或BCD码表示
- 整形运算
 - 算术运算：无符号整数、有符号整数
 - 按位运算：“|”、“&”、“~”、“^”
 - 逻辑运算：“||”、“&&”、“!”
 - 移位运算：“<<”、“>>”（逻辑移位、算术移位）
 - 位扩展和位截断：无符号和有符号